# HelloRPC 通信协议定义

George GeorgeWorld@qq.com | 2015

## 目录

1 报文结构 ....3
1.1 报文整体结构 ....3
1.2 TPDU 段详解 ....3
1.3 时间戳(Time stamp)段详解 ....3
1.4 报文头(Header)段详解 ....4
1.4.1 Header Field 1 报文头本身的长度 ....4
1.4.2 Header Field 2 报文头标识和报文版本号 ....4
1.4.3 Header Field 3 整个报文的总长度 ....5
1.4.4 Header Field 4 报文数据规则 ....5
1.4.5 Header Field 5 保留使用报文头数据段 ....5
1.4.6 Header Field 6 响应码 ....5
1.5 消息类型(MTI)段详解 ....5
1.6 固定含义域开关位图段(BitMap) ....6
1.7 固定含义域数据段(Definited Fields)详解 ....6
1.7.1 Definited Field 0 响应时间戳域 ....6
1.7.2 Definited Field 1 错误内容描述域 ....7
1.7.3 Definited Field 2 保留扩展域 ....7
1.7.4 Definited Field 3 保留扩展域 ....7
1.7.5 Definited Field 4 保留扩展域 ....7
1.7.6 Definited Field 5 保留扩展域 ....8
1.7.7 Definited Field 6 保留扩展域 ....8
1.7.8 Definited Field 7 MAC 校验域 ....8
1.1 消息内容(Msg-Content)详解 ....8
2 技术实现 ....9
3 附录 A：MAC 校验算法 ....10
4 附录 B：HelloRPC 参数类型 ....13

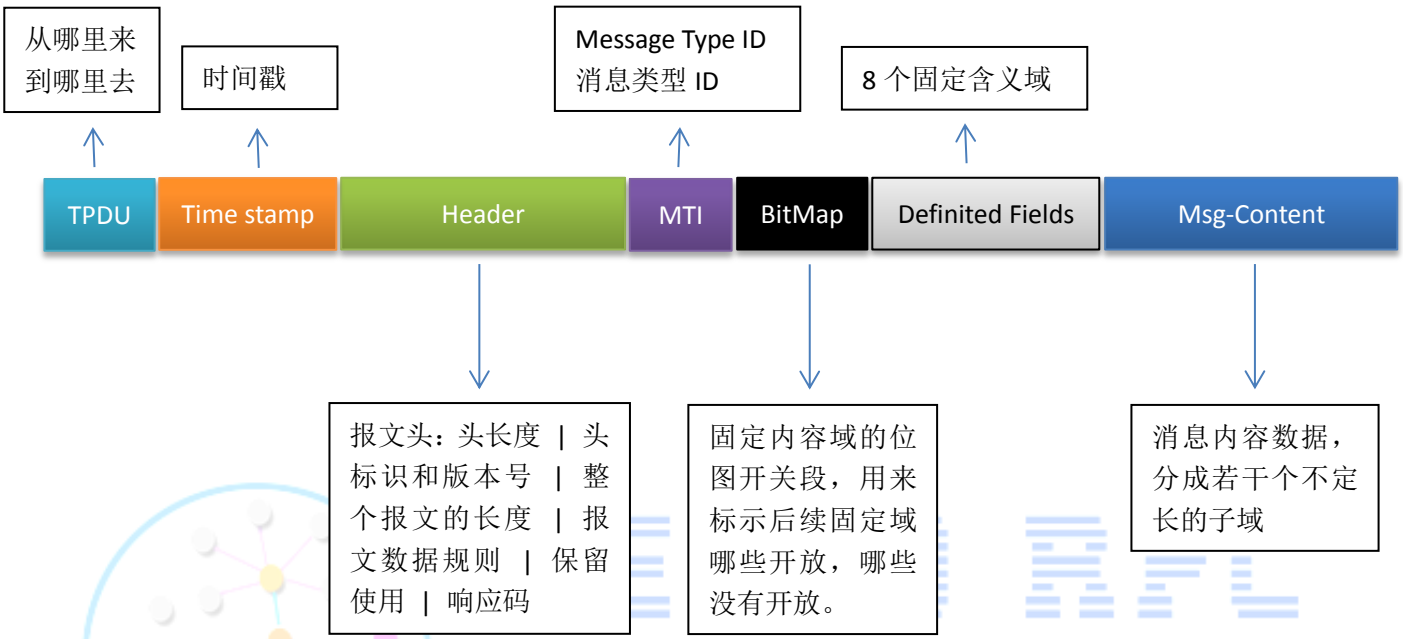
HELLO RPC
你好，远程过程调用框架
A small toy for experimenting,just for fun.

<table>
<tr><td>文档名称</td><td colspan="3">HelloRPC 通信协议定义</td></tr>
<tr><td>文档版本</td><td colspan="3">V1.0</td></tr>
<tr><td>创建人</td><td colspan="3">George <Georgeworld@qq.com></td></tr>
<tr><td>创建时间</td><td colspan="3">2015-11-16</td></tr>
<tr><td rowspan="3">更新记录</td><td>2015-11-16</td><td>George</td><td>创建文档</td></tr>
<tr><td>2015-11-19</td><td>George</td><td>精简报文内容段数据结构；<br>报文头中增加“报文数据规则”域；<br>更高报文头中“响应码”域的数据类型。</td></tr>
<tr><td>2015-11-24</td><td>George</td><td>补充固定含义扩展域的数据类型说明</td></tr>
</table>

HELLO RPC
你好，远程过程调用框架
A small toy for experimenting,just for fun.

# 1 报文结构

## 1.1 报文整体结构




你好，远程过程调用框架
A small toy for experimenting,just for fun.

## 1.2 TPDU 段详解

长度为 10 个字节，ASCII 数字字符，压缩传输时，使用右靠[1]BCD 码压缩为 5 个字节长度的数字字符。

| 60H | 目的地址数字标识 | 源地址数字标识 |
|---|---|---|
| 固定数字：60 | NNNN，四个字节的数字 | NNNN 四个字节的数字 |

数据实例：0x06 0x00 0x00 0x00 0x00 0x03 0x00 0x00 0x00 0x00

## 1.3 时间戳(Time stamp)段详解

长度为 13 个字节的数字，压缩传输时，使用 BCD 码压缩为 7 个字节长度的数字值（奇数个数字 BCD 压缩时，会左补数字 0）。

[1] 把数字转换为 8421BCD 码后，因为会用半个字节表示一个十进制数字，所以当数字个数为奇数个时，会出现最后组装的 BCD 数据，有半个字节的情况，这时候，将整个字节组靠右对齐，在字节组的最前面补 4 个 bit 位的 0，将最前面的半个字节，凑成一个字节，因为字节数组第一个字节，左补的 4 个 bit 的 0，按照 BCD 编码规则还原成数字以后，是数字 0，跟后面正常组装出来的数字组合在一起，成为 0123456789 这样的数字，而数字前面有 0，并不会影响数值的大小，所以用来标示数值的 BCD 码，都是采用右靠左补 0 的方式，来处理半个字节的情况。

## 1.4 报文头(Header)段详解

长度不固定，因为报文头中有描述报文长度的子域，而报文长度是不固定的，所以报文头长度也不能固定。

| 报文头长度 | 头标识和版本号 | 报文总长度 | 报文约束 | 保留使用 | 响应码 |
|---|---|---|---|---|---|
| 数字，长度不固定，根据第一个字节的第 1 个 Bit 位，确定是否有后续字节的长度描述 | 一个字节，8bit 的二进制数。 | 数字，长度不固定，根据第一个字节的第 1 个 Bit 位，确定是否有后续字节的长度描述。包括从 TPDU 到报文内容的整个报文的总长度。 | 一个字节长度，8bit 的二进制数。 | 2 个字节的二进制数据(byte[] 内容需要是数字)，保留作为扩展使用。 | 字符串，5 个字节长度，只支持 ASCII 码字符。 |
| Header Field 1 | Header Field 2 | Header Field 3 | Header Field 4 | Header Field 5 | Header Field 6 |

### 1.4.1 Header Field 1 报文头本身的长度

不包括 TPDU、时间戳、MTI、位图段和报文内容的长度，只包括 header 报文头本身的长度。

数字，长度不固定，按照【长度自描述】的规则进行组装，解析规则为如下。

先读取第一个字节，转换为二进制数，然后判断二进制数的第 8 位(bit8)，如果第 8 位是 0，则这个字节剩余的 7 个 bit 位转换为数字（最大 127dec），作为报文头的长度值。

如果第 8 位是 1，则将后续 7 个 bit 位转换为数字 n，表示后续还有 n 个字节用作长度描述。

比如读到的第一个字节是 0x82，转换为二进制数后是：10000010，判断 bit8 是 1，说明还有后续字节用作长度描述，所以就把剩余的 7 个 bit 位，转换为数字：00000010b=2dec，说明后续还有 2 个字节用作长度描述，则接着读取后面的两个字节，比如读到的第二字节是：0x01，第三个字节是：0x66，这两个字节分别转换为二进制数后是： 00000001 和 1100110，拼接在一起是：11100110，转换为十进制数是：230dec，所以报文头的长度是：230 个字节，当然，通常情况下，报文头不会这么长。

再比如，读取到的第一个字节是：0x16，转换为二进制数后是：00010110，第 8 个 bit 位是 0，说明第一个字节本身已经足够描述报文头的长度了，所以就把第一个字节的剩余 7 个 bit 位，转换为数字：00010110b=22dec，说明报文头的长度是 22 个字节。

### 1.4.2 Header Field 2 报文头标识和报文版本号

8bit 的二进制数。

第 8 个 bit 位的含义如下：

- 0 表示该报文是一个生产报文
- 1 表示该报文是一个测试报文

该 bit 位由通信发起方填写，在报文传输过程中保持不变（回复方保持该位不变）。

后 7 位二进制数，定义了该报文的版本，从 0000010 到 1111111，二进制数 0000001，作为保留版本号，用作以后扩展用。

也就是，可以正常用作协议版本号的十进制数字，是从 2dec – 127dec 126 个版本号可用。

### 1.4.3 Header Field 3 整个报文的总长度

包括从 TPDU 到报文内容的总长度。

数字，长度不固定，按照【长度自描述】的规则进行组装，具体解析规则见“**Header Field 1 报文头本身的长度**”章节。

### 1.4.4 Header Field 4 报文数据规则

一个字节长度，8bit 的二进制数。

各个 bit 位的含义如下：

1) **bit8** ：是否开启数据压缩，0=不压缩，1=压缩；
2) **bit7** ：是否开启数据加密，0=不加密，1=加密；
3) **bit6-bit1** ：尚未启用，备用 bit 位。

你好，远程过程调用框架
A small toy for experimenting,just for fun.

### 1.4.5 Header Field 5 保留使用报文头数据段

2 个字节(24bit)的二进制数。

未启用该数据段时，内容填充 0x00，也就是使用 3 个空字节填充（new byte[3]）

### 1.4.6 Header Field 6 响应码

5 个字节的定长字符串，只支持 ASCII 码字符。

具体响应码的含义，见下文中，关于“响应码”的章节。

## 1.5 消息类型(MTI)段详解

4 个字节的数字 ASCII 字符，压缩时使用 BCD(非偶数字个数时，左补 0)码压缩为 2 个字节来表示。

MTI 是 Message Type ID 的简写，也就是报文类型 ID。

具体的 MTI 定义如下：

| MTI | 含义 | 备注 |
|---|---|---|

| | | |
|---|---|---|
| 001 | 待定义 | |
| 002 | 待定义 | |
| 003 | 待定义 | |
| 004 | 待定义 | |
| 005 | 待定义 | |
| 006 | 待定义 | |
| 007 | 待定义 | |
| 008 | 待定义 | |
| 009 | 待定义 | |
| 010 | 待定义 | |
| 011 | 待定义 | |

## 1.6 固定含义域开关位图段(BitMap)

固定一个字节的二进制位(bit8)构成的位图，用来描述后续的 8 个固定含义域的开关情况。

域序号从 0 开始，序号排序方向，与字节的 bit 位序号排序方向相反。

比如，一个字节的二进制值是：01100000，那么这个字节的 bit 位序与固定含义域的域序号对应关系如下：

| 字节内容 | **0** | **1** | **1** | **0** | **0** | **0** | **0** | **0** |
|---|---|---|---|---|---|---|---|---|
| Bit 位序 | bit8 | bit7 | bit6 | bit5 | bit4 | bit3 | bit2 | bit1 |
| 固定含义域序号 | Field 0 | Field 1 | Field 2 | Field 3 | Field 4 | Field 5 | Field 6 | Field 7 |

比如：位图字节是 0x60，转换成二进制数是：01100000，表示位图后面的固定含义域数据段内，只有第 1 域，第 2 域的数据，其他 6 个固定含义域没有数据，也就是固定含义域数据段内，数据的排列方式是：

| 第 **1** 域数据 | 第 **2** 域数据 |
|---|---|

如果位图字节的二进制数是：00000010，则表示后面的固定含义域数据段内，只有第 6 域的数据。

| 第 6 域数据 |
|---|

BitMap 位图段是固定长度的定义段，占一个字节，分别对应 8 个固定含义数据域。

## 1.7 固定含义域数据段(Definited Fields)详解

固定含义域，指域内容的含义确定不变，不是指域内容的数据长度固定不变。

目前有 8 个固定含义域，分别定义如下：

### 1.7.1 Definited Field 0 响应时间戳域

服务端响应时间的时间戳，是由 13 个数字组成的 ASCII 字符。

| 域长度类型 | 数据长度 | 数据属性 | BCD 压缩 |
| --- | --- | --- | --- |
| 定长域 | 13 个字节 | 数字 | 使用右靠左补 BCD 压缩时，用 7 个字节表示 13 个字节的数字 ASCII 字符 |

### 1.7.2 Definited Field 1 错误内容描述域

当服务端的响应发生错误时，把错误的详细描述，已 **UTF-8** 编码的方式，放入本域内。

| 域长度类型 | 数据长度 | 数据属性 |
| --- | --- | --- |
| 不定长域 | 3 个字节的长度值 + 最大 512 个字节的数据域 | UTF-8 字符串 |

### 1.7.3 Definited Field 2 保留扩展域

保留扩展域

| 域长度类型 | 数据长度 | 数据属性 |
| --- | --- | --- |
| 不定长域 | 3 个字节的长度值 + 最大 512 个字节的数据域 | Byte[]二进制数据 |

### 1.7.4 Definited Field 3 保留扩展域

你好，远程过程调用框架
A small tool for experimenting just for fun

保留扩展域

| 域长度类型 | 数据长度 | 数据属性 |
| --- | --- | --- |
| 不定长域 | 3 个字节的长度值 + 最大 512 个字节的数据域<br><br>扩展使用时，如果约定使用 BCD 压缩传输**数字字符**，则该域数据部分会被压缩至原数据的一半。 | Byte[]二进制数据 |

### 1.7.5 Definited Field 4 保留扩展域

保留扩展域

| 域长度类型 | 数据长度 | 数据属性 |
| --- | --- | --- |
| 不定长域 | 3 个字节的长度值 + 最大 512 个字节的数据域<br><br>扩展使用时，如果约定使用 BCD 压缩传输**数字字符**，则该域数据部分会被压缩至原数据的一半。 | Byte[]二进制数据 |

### 1.7.6 Definited Field 5 保留扩展域

保留扩展域

| 域长度类型 | 数据长度 | 数据属性 |
|---|---|---|
| 不定长域 | 3 个字节的长度值 + 最大 512 个字节的数据域<br><br>扩展使用时，如果约定使用 BCD 压缩传输**数字字符**，则该域数据部分会被压缩至原数据的一半。 | Byte[]二进制数据 |

### 1.7.7 Definited Field 6 保留扩展域

保留扩展域

| 域长度类型 | 数据长度 | 数据属性 |
|---|---|---|
| 不定长域 | 3 个字节的长度值 + 最大 512 个字节的数据域<br><br>扩展使用时，如果约定使用 BCD 压缩传输**数字字符**，则该域数据部分会被压缩至原数据的一半。 | Byte[]二进制数据 |

### 1.7.8 Definited Field 7 MAC 校验域

报文 MAC 校验域，用来存放整个报文的 MAC 校验值(**不包括本域数据**)，防止报文被中途篡改。
MAC 算法见“**附录 A**”。

| 域长度类型 | 数据长度 | 数据属性 |
|---|---|---|
| 定长域 | 8 个字节(bit64) | byte[]二进制数据 |

## 1.1 消息内容(Msg-Content)详解

消息内容部分，使用标准的 TLV 数据格式，来表示远程方法结构，如果远程方法的入参是对象类型，则使用标准 Java 序列化(Java Sequence)后的对象二进制数据格式，详细的 Java 序列化数据格式，见文档《Java 序列化的机制和原理》（Java 序列化的机制和原理.pdf）

**消息内容，采用嵌套 TLV 格式，图示如下：**

| | |
|---|---|
| Tag | 动作类命名空间或包路径 tag |
| Length | 动作类命名空间或包路径长度 |
| Value | 动作类命名空间或包路径的值 |
| | |
| Tag | 动作类名称 Tag |

你好，远程过程调用框架
A small toy for experimenting,just for fun.

| | | | |
|---|---|---|---|
| Length | 动作类名称长度 | | |
| Value | 动作类名称内容 | | |
| | | | |
| Tag | 方法返回值类型 tag | | |
| Length | 方法返回值类型长度 | | |
| Value | 方法返回值类型数据 | | |
| | | | |
| Tag | 方法返回值数据 tag | | |
| Length | 方法返回值数据长度 | | |
| Value | 方法返回值数据对象（或基本类型） | | |
| | | | |
| Tag | 方法名称 tag | | |
| Length | 方法名称长度 | | |
| Value | 方法名称数据 | | |
| | | | |
| Tag | 方法参数列表 tag | | |
| Length | 方法参数列表长度 | | |
| Value | Tag | 第一个参数 Tag | |
| | Length | 第一个参数长度 | |
| | Value | Tag | 方法参数类型 Tag |
| | | Length | 方法参数类型长度 |
| | | Value | 方法参数类型数据<br>参数类型，见**附录 B** |
| | | | |
| | | Tag | 方法参数值 Tag |
| | | Length | 方法参数值长度 |
| | | Value | 方法参数值数据对象（或基本类型）<br>如果是对象参数，则这里的数据是标准的 Java 序列化数据格式 |

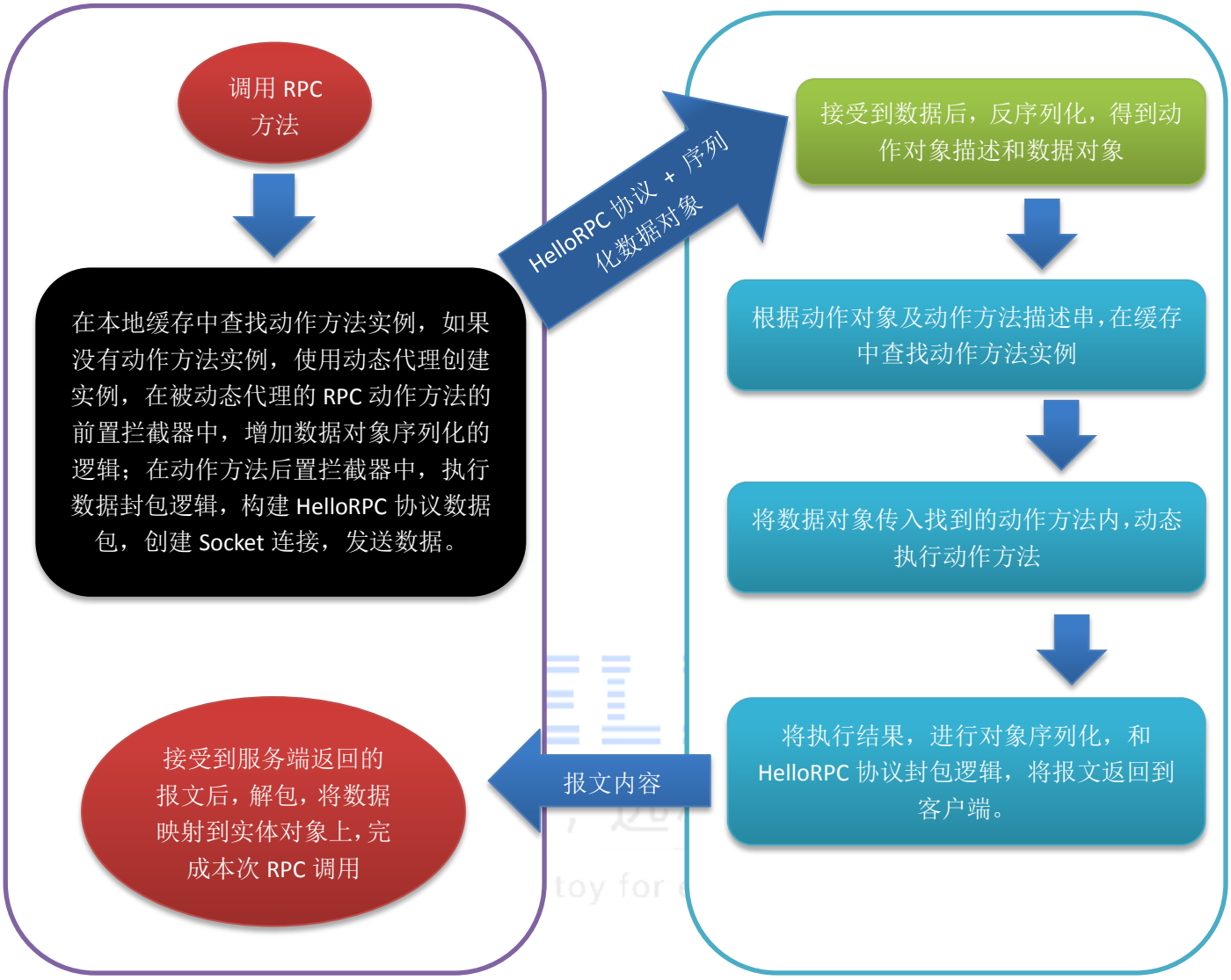
# 2 技术实现

## 《客户端》

Java 语言

## 《服务端》

Java/Qt C++ 语言




A. 要考虑同步调用和异步调用两种调用模式；
B. 要尽量做到对象缓存化，无论是在客户端，还是在服务端。

# 3 附录 A：MAC 校验算法

HelloRPC 报文 MAC 的算法

HelloRPC 采用 E C B 的加密方式，简述如下：

a) 将欲发送给 POS 中心的消息中，从消息类型（MTI）

到 63 域 之 间 的 部 分 构 成 MAC

ELEMEMENT BLOCK （MAB）。

b) 对 MAB，按每 8 个字节做异或（不管信息中的字符格式），如果最后不满 8 个字节，则添

加“0X00”。

示例 ：

MAB = M1 M2 M3 M4

其中：

M1 = MS11 MS12 MS13 MS14 MS15 MS16 MS17 MS18

M2 = MS21 MS22 MS23 MS24 MS25 MS26 MS27 MS28

M3 = MS31 MS32 MS33 MS34 MS35 MS36 MS37 MS38

M4 = MS41 MS42 MS43 MS44 MS45 MS46 MS47 MS48

按如下规则进行异或运算：

MS11 MS12 MS13 MS14 MS15 MS16 MS17 MS18

XOR） MS21 MS22 MS23 MS24 MS25 MS26 MS27 MS28

---

TEMP BLOCK1 = TM11 TM12 TM13 TM14 TM15 TM16 TM17 TM18

然后，进行下一步的运算：

TM11 TM12 TM13 TM14 TM15 TM16 TM17 TM18

XOR） MS31 MS32 MS33 MS34 MS35 MS36 MS37 MS38

---

TEMP BLOCK2 = TM21 TM22 TM23 TM24 TM25 TM26 TM27 TM28

再进行下一步的运算：

HELLO RPC
你好，远程过程调用框架
A small toy for experimenting,just for fun.

TM21 TM22 TM23 TM24 TM25 TM26 TM27 TM28

XOR） MS41 MS42 MS43 MS44 MS45 MS46 MS47 MS48

---

RESULT BLOCK = TM31 TM32 TM33 TM34 TM35 TM36 TM37 TM38

c) 将异或运算后的最后 8 个字节（RESULT BLOCK）转换成 16 个 HEXDECIMAL：

RESULT BLOCK = TM31 TM32 TM33 TM34 TM35 TM36 TM37 TM38

= TM311 TM312 TM321 TM322 TM331 TM332 TM341 TM342 ||

TM351 TM352 TM361 TM362 TM371 TM372 TM381 TM382

d) 取前 8 个字节用 MAK 加密：

ENC BLOCK1 = eMAK（TM311 TM312 TM321 TM322 TM331 TM332 TM341 TM342）

你好，远程过程调用框架
A small toy for experimenting,just for fun.





= EN11 EN12 EN13 EN14 EN15 EN16 EN17 EN18

e) 将加密后的结果与后 8 个字节异或：

EN11 EN12 EN13 EN14 EN15 EN16 EN17 EN18

XOR） TM351 TM352 TM361 TM362 TM371 TM372 TM381 TM382

---

TEMP BLOCK= TE11 TE12 TE13 TE14 TE15 TE16 TE17 TE18

f) 用异或的结果 TEMP BLOCK 再进行一次单倍长密钥算法运算。

ENC BLOCK2 = eMAK（TE11 TE12 TE13 TE14 TE15 TE16 TE17 TE18）

= EN21 EN22 EN23 EN24 EN25 EN26 EN27 EN28

g) 将运算后的结果（ENC BLOCK2）转换成 16 个 HEXDECIMAL：

ENC BLOCK2 = EN21 EN22 EN23 EN24 EN25 EN26 EN27 EN28

= EM211 EM212 EM221 EM222 EM231 EM232 EM241 EM242 ||

EM251 EM252 EM261 EM262 EM271 EM272 EM281 EM282

示例 ：

ENC RESULT= %H84, %H56, %HB1, %HCD, %H5A, %H3F, %H84, %H84

转换成 16 个 HEXDECIMAL:

“8456B1CD5A3F8484”

h) 取前 8 个字节作为 MAC 值。

取”8456B1CD”为 MAC 值。

# 4 附录 B：HelloRPC 参数类型

HelloRPC 的类型定义，基本上与 google protobuf 相同（精简了不常用的数据类型，增加了对象类型），详细定义如下：

HelloRPC 类型定义表

| HelloRPC Type | 简写 | 说明 | C++ Type | Java Type | Go Type |
|---|---|---|---|---|---|
| double | d | | double | double | float64 |
| float | f | | float | float | float32 |
| int32 | i32 | Uses variable-length encoding. Inefficient for encoding negative numbers – if your field is likely to have negative values, use sint32 instead. | Int32 | int | int32 |
| int64 | i64 | Uses variable-length encoding. Inefficient for encoding negative numbers – if your field is likely to have negative values, use sint64 instead. | Int64 | long | int64 |

| | | | | | |
|---|---|---|---|---|---|
| bool | b | | bool | boolean | bool |
| string | s | A string must always contain UTF-8 encoded or 7-bit ASCII text. | String | String | string |
| bytes | bt | May contain any arbitrary sequence of bytes. | String | ByteString | []byte |
| object | o | 对象类型 | Java 对象 | | |

HELLO RPC
你好，远程过程调用框架
A small toy for experimenting,just for fun.